

Performance, Escalabilidade e Testes

Arthur Souza e João Victor Prata
Fundação Getúlio Vargas

Performance

Performance

*Na grande maioria das vezes, o grande vilão da performance
está associado ao **Change Detection!***

Performance

- OnPush
- Lazy Loading
- Deferred loading
- Tree Shaking
- Mecanismos de Cache
- Boas práticas gerais
- Como avaliar a Performance (Performance Profiler)

OnPush

A **estratégia de detecção de mudanças OnPush** otimiza o desempenho de uma aplicação ao não obrigatoriamente verificar os componentes filhos, mesmo que tenha verificado o componente pai.

- A **estratégia padrão** de detecção de alterações funciona de modo que **um componente filho sempre será verificado** se o seu componente pai for verificado.
- A estratégia OnPush instrui o framework a **verificar mudanças** em um componente **apenas quando ocorrem eventos específicos (Inputs alterados, Markforcheck)**, em vez de monitorar todas as interações da aplicação.
- Assim, caso um componente pai for verificado, **não necessariamente** o componente filho será verificado também

```
@Component({  
  selector: 'app-dashboard',  
  standalone: true,  
  imports: [CommonModule, ScrollingModule],  
  templateUrl: './dashboard.html',  
  changeDetection: ChangeDetectionStrategy.OnPush  
})
```

Impacto:

- Menor número de ciclos de detecção;
- Reduz Change Detection em até 90%

Lazy Loading

Lazy Loading é uma técnica de otimização que consiste em carregar partes da sua aplicação apenas no momento em que elas são **realmente necessárias**, em vez de **carregar tudo de uma vez no início**.

- A estratégia é **dividir a aplicação** em módulos ou grupos de componentes, de modo que o código de uma página **só é baixado quando o usuário clica nela**

```
const routes: Routes = [
  {
    path: 'dashboard',
    loadComponent: () => import('./dashboard/dashboard.component')
      .then(m => m.DashboardComponent) // 👉 Só carrega quando acessado
  }
];
```

Impacto:

- O usuário **não precisa esperar** o download de 2MB de código para ver uma tela que usa apenas 50KB.
- O app principal fica **leve e rápido**. O dashboard só carrega quando o usuário realmente precisa.

Deferred loading

Deferred loading (@defer) também é uma técnica de otimização, que permite carregar pedaços de um componente apenas quando certas condições ocorrerem:

- **on viewport:** Carrega o componente (ex: um gráfico pesado) apenas quando **ele aparece na tela durante o scroll**.
- **on idle:** Carrega quando o navegador estiver "**descansando**", ou seja, **terminou de carregar o essencial da página**.
- **on interaction:** Carrega apenas se o usuário **clicar em um botão específico**.
- **on timer(5s):** Carrega após um tempo determinado.

O @defer já traz também nativamente o **gerenciamento de estados**:

- **@placeholder:** O que mostrar antes (um **esqueleto**/loading).
- **@loading:** O que mostrar enquanto o arquivo **está sendo baixado**.
- **@error:** O que mostrar se a **internet cair** durante o carregamento, por exemplo.

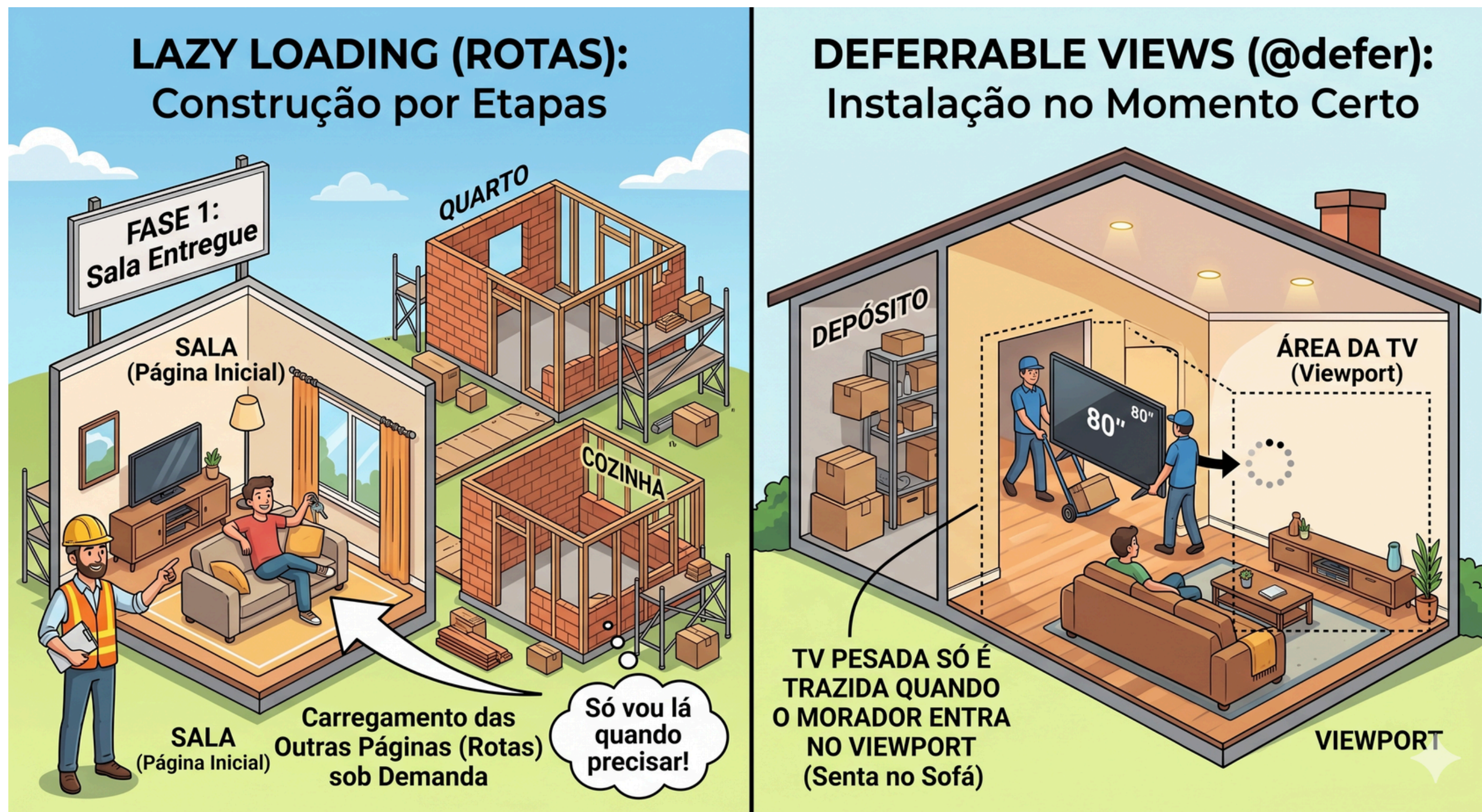
Deferred loading

Impacto:

- **Redução do Tempo de Interatividade (TTI):** O site fica "clicável" muito mais rápido;
- **Economia de Dados:** Usuários não baixam o que não vão usar;

```
</section>
@defer (on timer(3s)) {
  <section class="bg-white/95 p-5 rounded-xl shadow-2xl h-[300px]">
    <h3 class="font-bold mb-6">📊 Distribuição Salarial</h3>
    <div class="flex items-end gap-6 h-40">
      @for (f of histograma(); track f.label) {
        <div class="flex-1 flex flex-col items-center justify-end h-full">
          <span class="text-xs mb-2">{{f.count}}</span>
          <div class="w-full bg-blue-500 rounded-t transition-all duration-500" [style.height.%]="f.percent"></div>
          <span class="text-[10px] mt-2">{{f.label}}</span>
        </div>
      }
    </div>
  </section>
} @placeholder {
  <div class="h-[300px] bg-white/50 rounded-xl animate-pulse flex items-center justify-center text-slate-400">Carregando...
</div>
```

Deferred x Lazy Loading



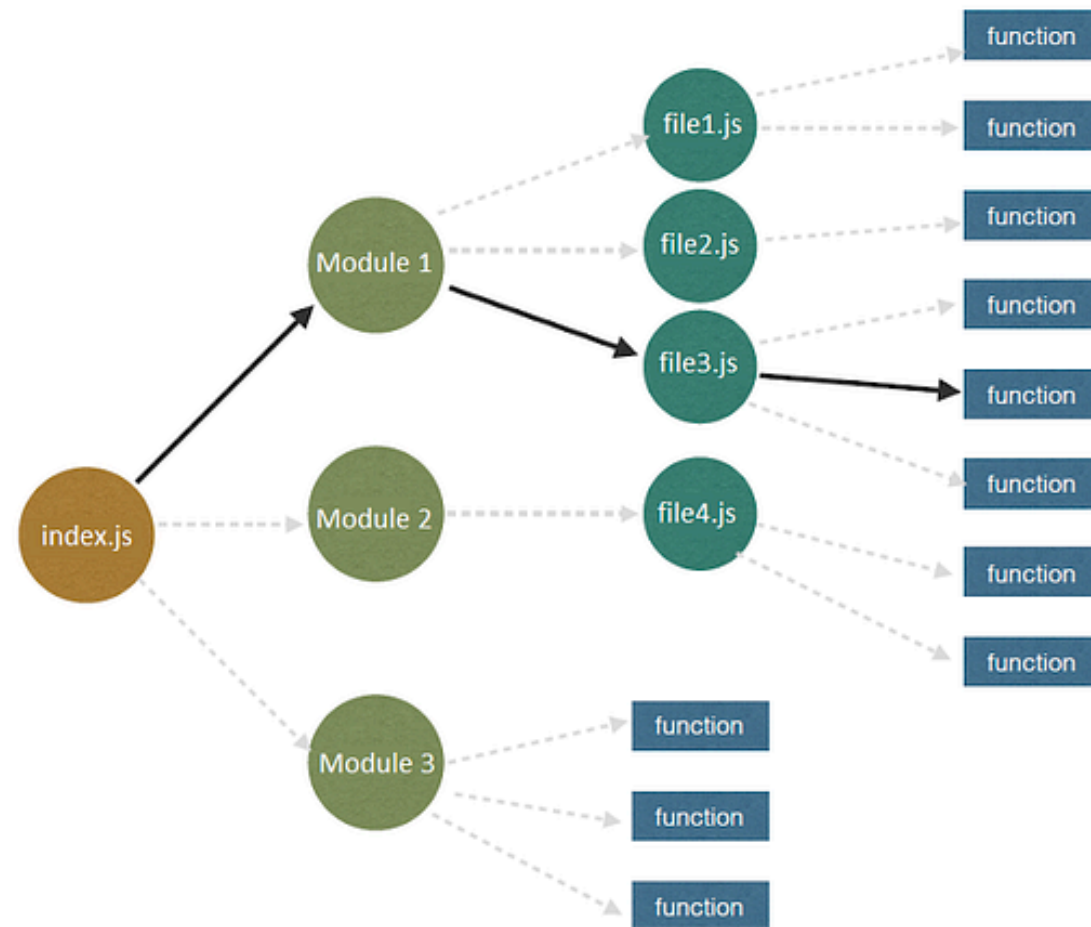
Tree Shaking

Tree Shaking é uma técnica de otimização cuja ideia base é **remover todo código não utilizado (dead code)**. Ele é realizado quando rodamos **ng build**, que faz o seguinte:

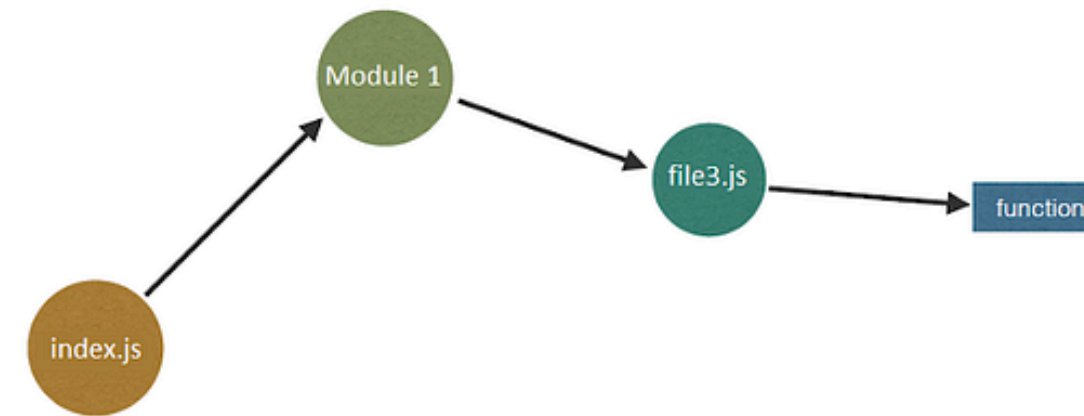
- **Leitura:** O compilador lê seus arquivos .ts;
- **Transpilação:** Ele converte o TypeScript para JavaScript;
- **Otimização:** Ele pega esse JavaScript gerado, remove o que não é usado (**Tree Shaking**) e encurta tudo (**minificação**);
- **Escrita:** Ele salva o resultado final em novos arquivos dentro da pasta dist/.

Tree Shaking

Before Tree Shaking



After Tree Shaking



Mecanismos de Cache

O Cache é uma **estratégia de gerenciamento de estado e otimização de performance**. Seu objetivo principal é evitar que a aplicação peça ao servidor a **mesma informação repetidas vezes** ou que o navegador **re-processe dados que já possui**.

- Ao realizar chamadas de API, as respostas delas não mudam com frequência. Nesses casos, podemos adicionar **um mecanismo de cache e armazenar o valor da API**.
- Quando for realizado outro pedido à mesma API, é **verificado se existe um valor para ele na cache**.
 - Se existir, ele é utilizado.
 - Caso contrário, realiza a chamada da API e coloca o resultado em cache.
- Um exemplo da utilização de cache seria por meio do operador de pipe **shareReplay()**, que serve para **armazenar o resultado de uma requisição e compartilhar esse resultado para as solicitações idênticas que forem feitas**.

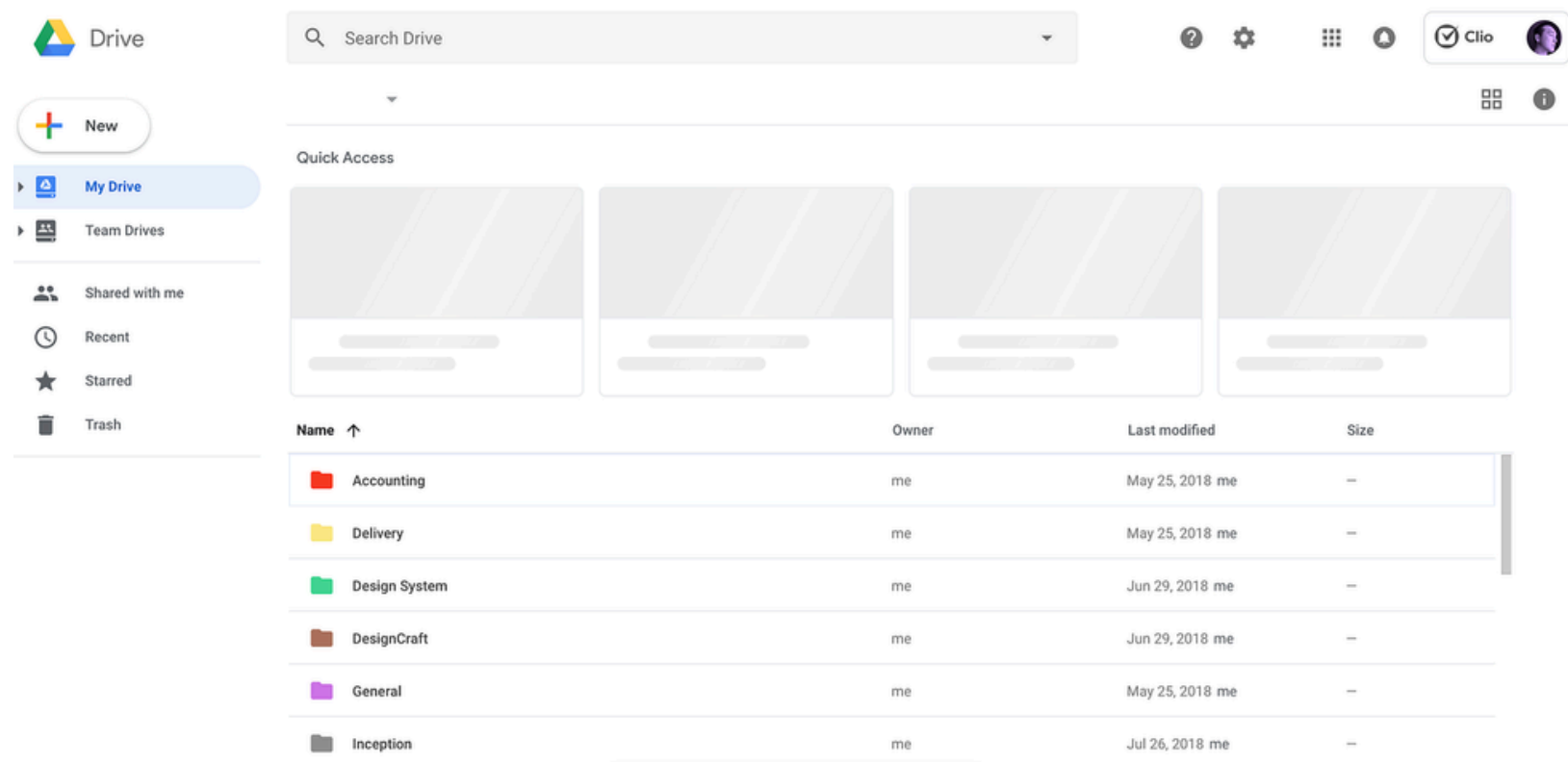
Boas Práticas Gerais - Performance

- **Signals:** Em vez de verificar a aplicação inteira quando uma mudança ocorre, já **sabemos o que depende de cada parte**, e **verificamos apenas o necessário** (quem depende daquele signal);
- **Track:** Antes, quando havia uma mudança, a **lista inteira tinha que ser renderizada novamente**. Utilizando o track, **apenas** os itens da lista que **realmente mudaram** são atualizados;
- **Unsubscribe:** Observables que **continuam emitindo** depois que o **componente foi destruído** são uma fonte comum de problemas, principalmente do **Memory Leak**. (takeUntil e takeUntilDestroyed);
- **Evitar ter subscrições dentro de subscrições;**

Boas Práticas Gerais - Performance

- **Implementar rolagem virtual para listas grandes:** A rolagem virtual **renderiza apenas os itens visíveis em uma lista**, em vez de criar elementos DOM para milhares de itens. Essa técnica melhora drasticamente o desempenho ao exibir grandes conjuntos de dados.
- **Priorizar o uso de Icon Fonts em vez de Icon importados individualmente:** Reduz o volume de arquivos baixados e torna o carregamento da página mais leve.
- **Entretenimento do usuário (UX):**
 - Em vez de um spinner girando ou uma tela vazia, **utilizar silhuetas (Skeleton)** que imitam a estrutura final da página
 - Sempre que o **usuário realizar uma ação pesada**, fornecer uma **resposta visual instantânea** (uma animação - **Lootie Web**, uma barra de progresso ou uma mudança de estado do botão).

Boas Práticas Gerais - Performance



- Silhuetas (Skeletons)



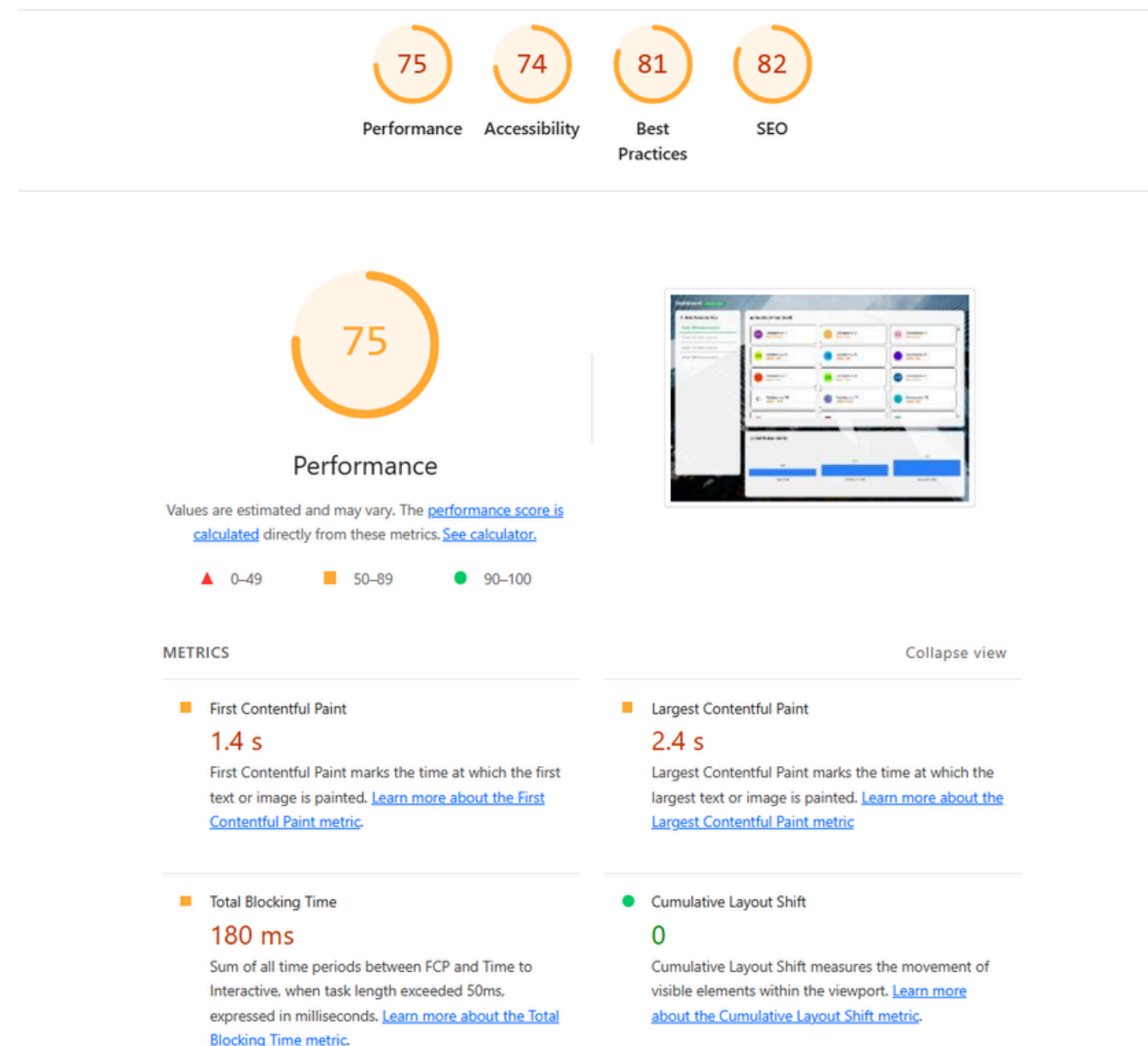
- Animações

Como avaliar a Performance?

- **Métricas de Performance:**
 - **LCP (Largest Contentful Paint):** Mede o **tempo de carregamento** do maior elemento visual da tela.
 - **Valor ideal:** Abaixo de 2.5 segundos.
 - **FCP (First Contentful Paint):** Mede o quão rápido o **primeiro bit de conteúdo aparece**.
 - **Valor ideal:** Abaixo de 1,8 segundos.
 - **CLS (Cumulative Layout Shift):** Mede a **estabilidade visual** da página.
 - **Valor ideal:** Abaixo de 0,1.
 - **TBT (Total Blocking Time):** Mede o tempo total em que a aplicação ficou "travada" e não **conseguiu responder à entrada do usuário (como um clique ou digitação)** durante o carregamento inicial.
 - **Valor ideal:** Abaixo de 200 milissegundos.
 - **Speed Index:** Mede o quão rápido o conteúdo da página é visualmente preenchido, ou seja, **o quão rápido fica com a aparência de “pronta”**.
 - **Valor ideal:** Abaixo de 3,4 segundos.

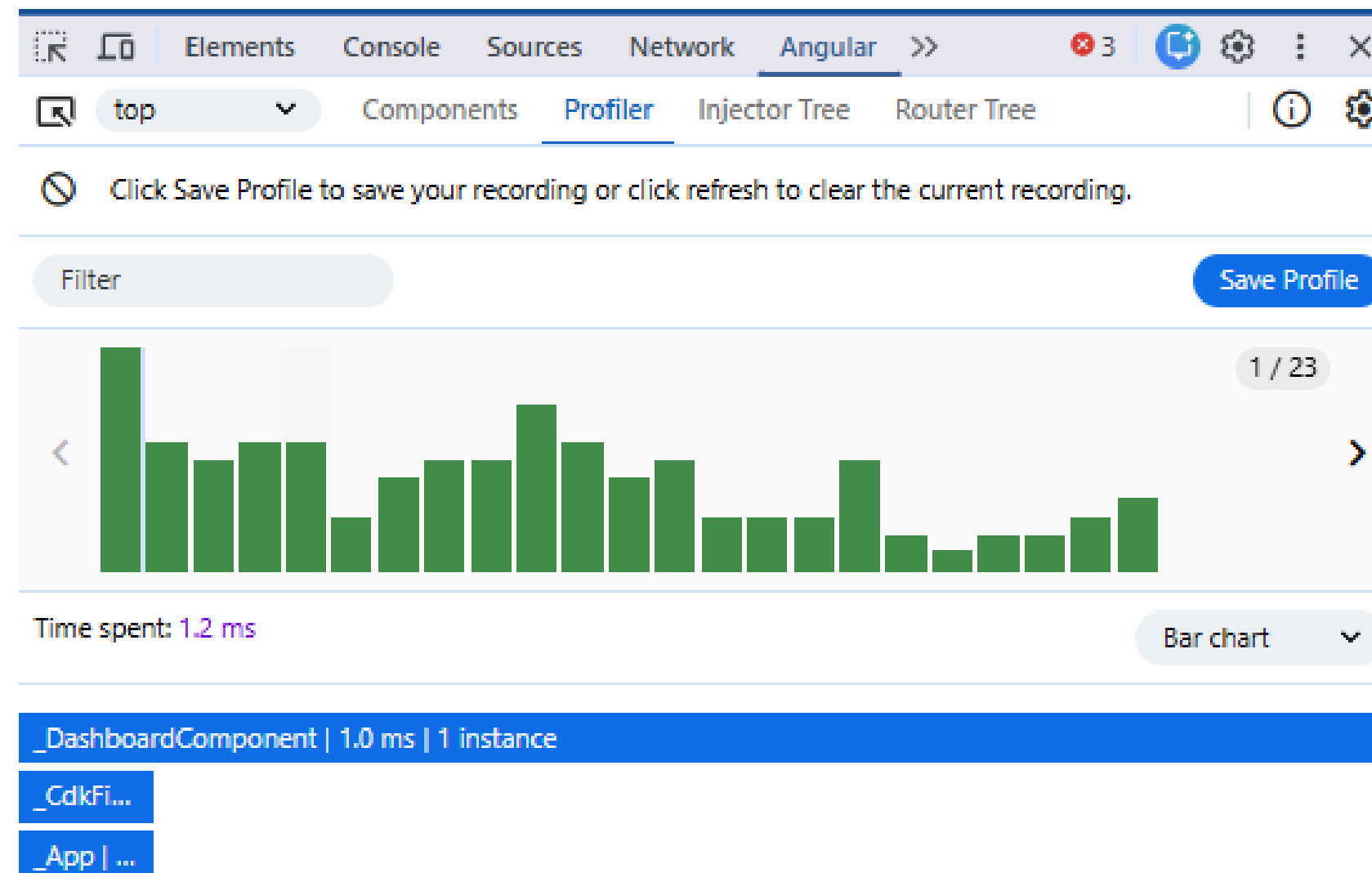
Como avaliar a Performance?

- Ferramentas de avaliação (Painel do desenvolvedor):
 - **Relatórios Lighthouse:** Realiza uma auditoria na aplicação e gera um **relatório consolidado** com as **principais métricas de performance**.



Como avaliar a Performance?

- Ferramentas de avaliação (Painel do desenvolvedor):
 - **Profiler (Angular DevTools):** Permite “gravar” a execução da aplicação e compreender **como está a execução do Change Detection**;
 - Permite analisar a métrica de duração de ciclo, que mede o tempo total que o Angular levou desde o gatilho até a finalização da verificação de mudanças. **Valor ideal:** Abaixo de 16ms



Escalabilidad

Escalabilidade

- Lazy Loading
- Tree Shaking
- Ícones
- Imports desnecessários
- Deferred Loading
- **CDNs**
- **Organização de pastas**

CDNs (Content Delivery Network)



Sem CDN, todos os usuários precisam baixar os arquivos do mesmo servidor.

CDNs distribuem cópias dos arquivos em servidores ao redor do mundo:

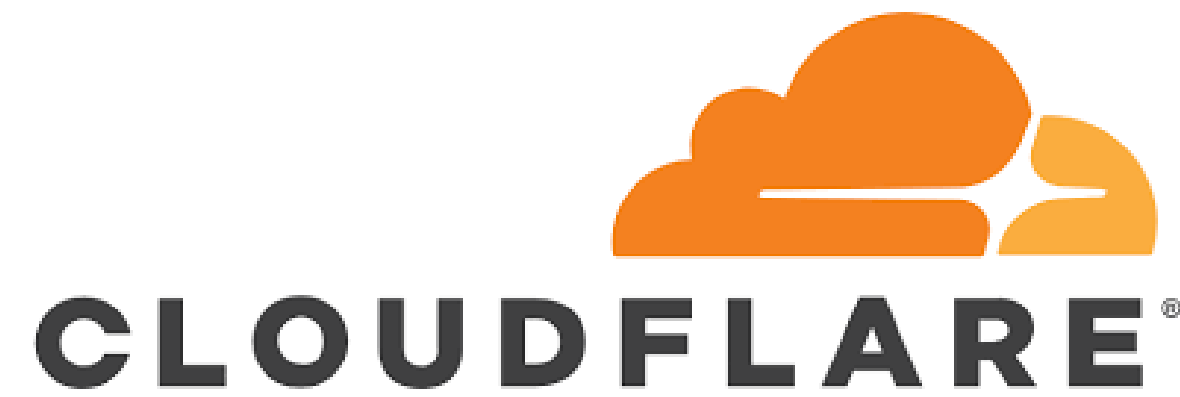
- CSS
- JS
- imagens
- fontes

são carregados do servidor mais próximo do usuário

A CDN reduz latência e evita sobrecarga em um único servidor, permitindo que a aplicação suporte mais usuários simultaneamente

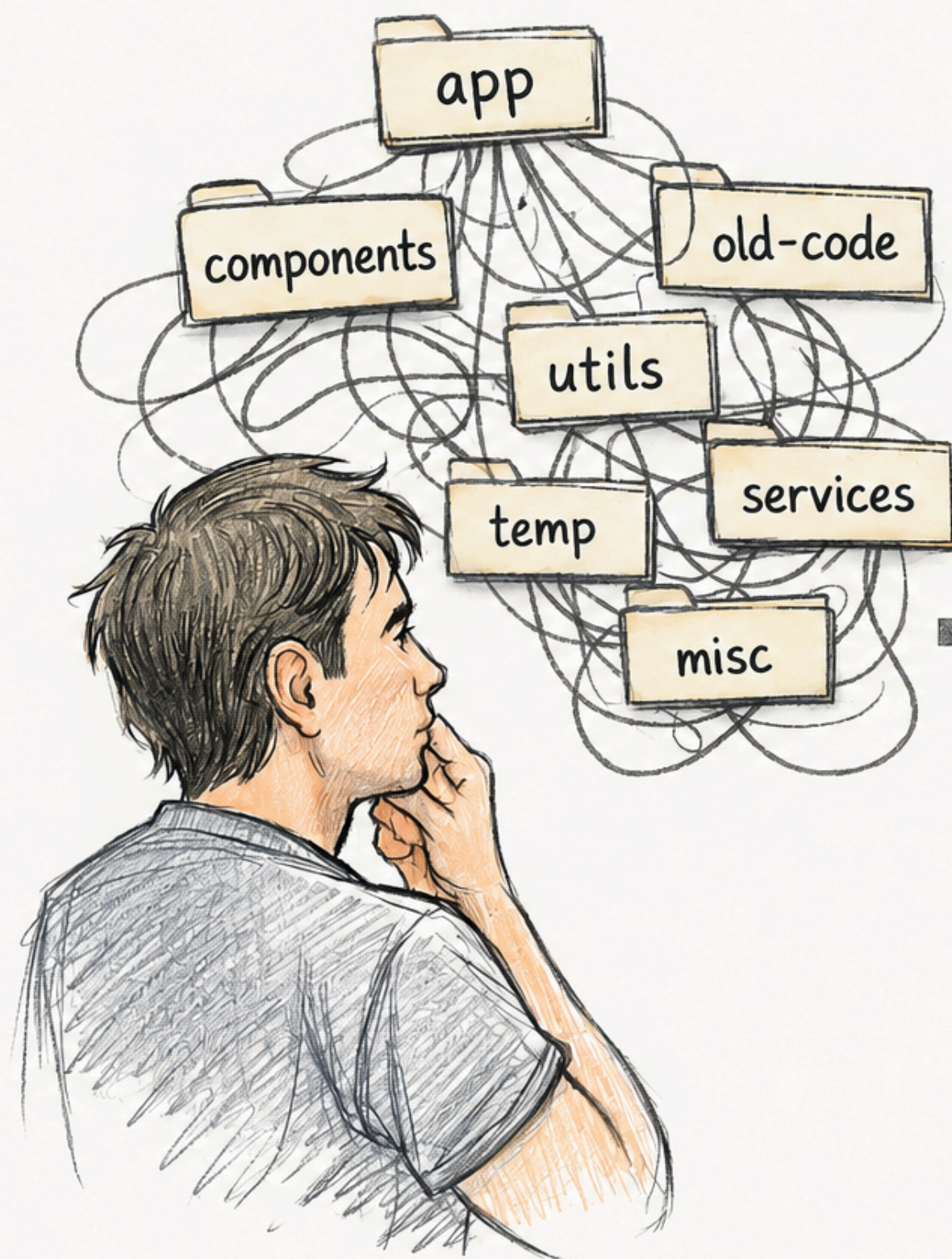
CDNs (Content Delivery Network)

Exemplos:

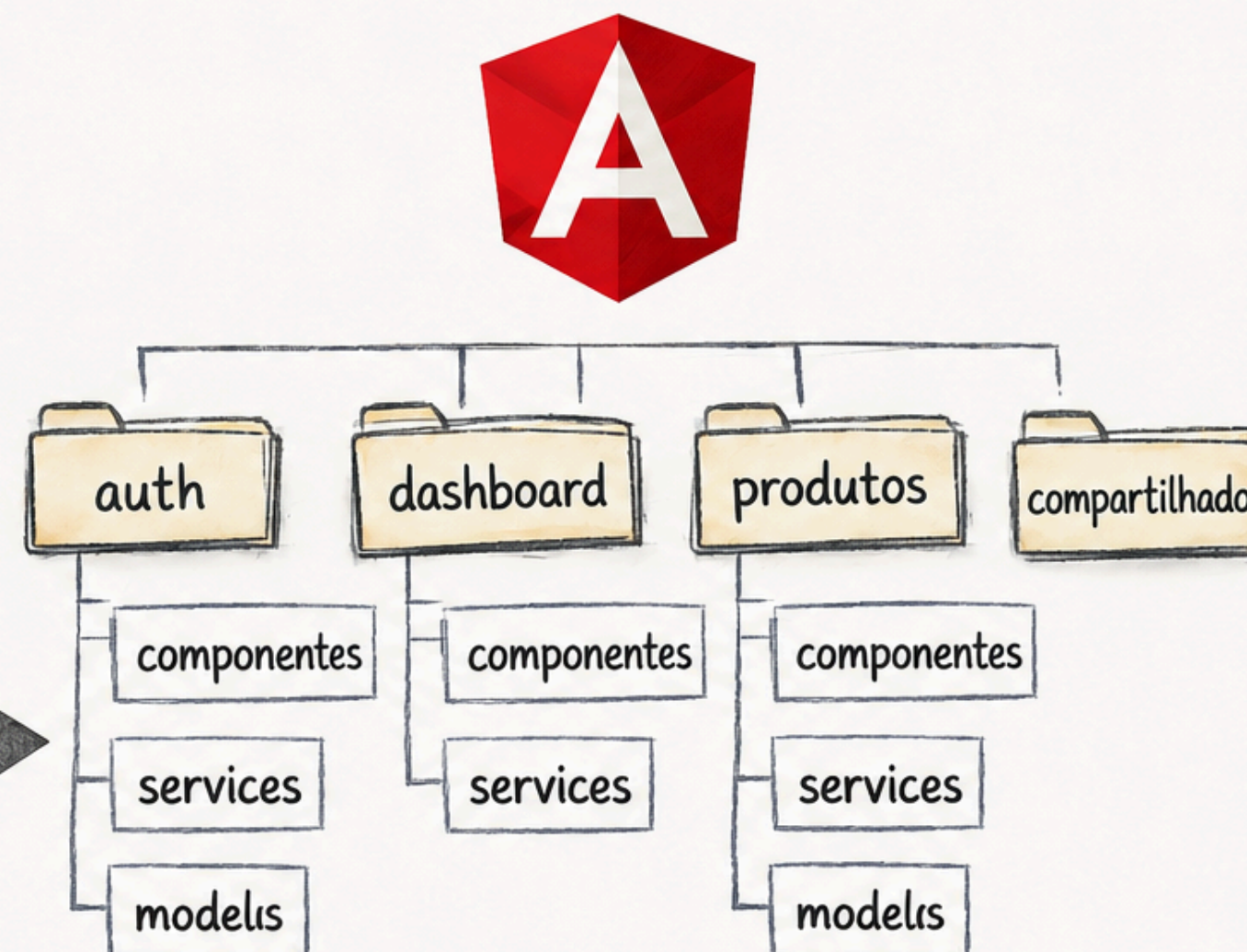


Estruturação de pastas

Estrutura de Pastas Desorganizada



Arquitetura Angular Modular



- Facilidade de navegação
- Facilidade de refatoramento
- Features como unidades independentes e reutilizáveis

Testes

Por que fazer testes?

O Component funciona
conforme o esperado?



FormComponent

(nome)="usuario"

O Component funciona
conforme o esperado?



BotaoSalvarComponent

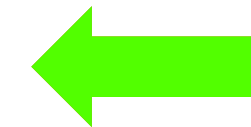
@Input() nome;

O Service funciona
conforme o esperado?

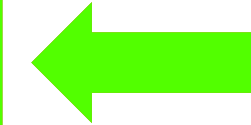


ApiService

O Input funciona
conforme o esperado?



O Injection funciona
conforme o esperado?



Por que fazer testes?

Proteger contra alterações
que quebrem o código

Analisar comportamentos do código
(esperado e não esperado)

Revelar erros de design

Teste unitário vs E2E (End-to-End)

Unitário

Testa:

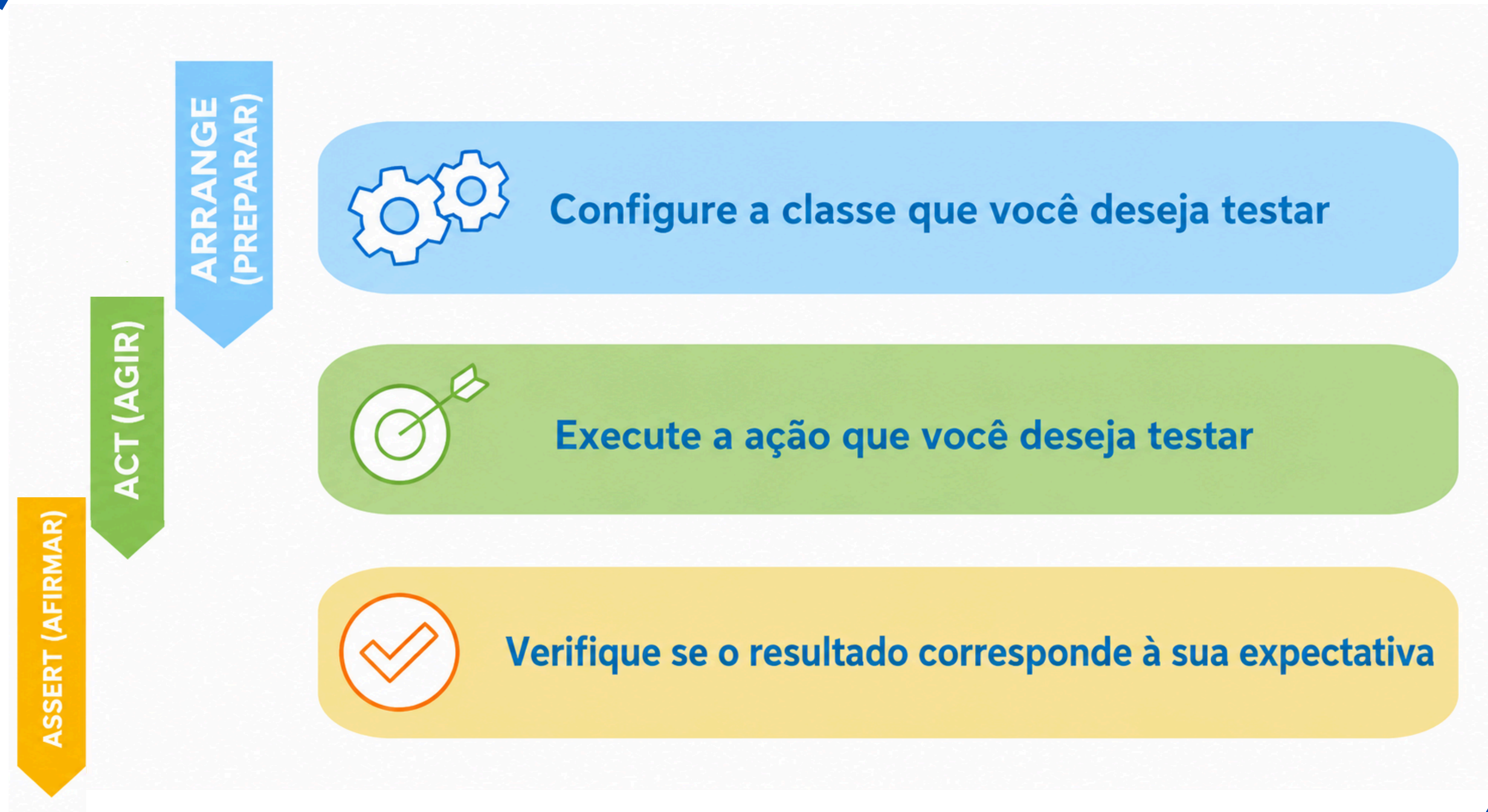
- função
- componente
- service

E2E

Testa:

- fluxo inteiro do usuário

AAA Pattern



O que NÃO testar

Evite testar

- Variáveis privadas
- Implementação interna
- Bibliotecas do Angular
- HTML extremamente específico
- Dependências externas reais (API/banco)

Prefira testar

- Comportamento visível
- Resultado esperado
- Interação do usuário
- Regras de negócio
- Estados importantes da tela

Q&A

Referências utilizadas

- **Course: Angular - The Complete Guide (Section 44: A Basic Introduction to Unit Testing in Angular Apps) - Maximilian Schwarzmüller**
(https://www.udemy.com/share/101WgA3@twAETpdOFol3u-DGqUH2MSzUeHrmHNwt9qlK8yIUF_IGSfLboKrWfCTwaCABa9J6QA==/)
- **9 Angular Folder Structure Mistakes That Slowly Destroy Your Codebase** (<https://medium.com/@sourabhda1998/9-angular-folder-structure-mistakes-that-slowly-destroy-your-codebase-27cc31b73df6>)
- **5 técnicas para melhorar a performance no seu app angular**
(<https://dev.to/dev-araujo/5-tecnicas-para-melhorar-a-performance-no-seu-app-angular-g0e>)

Referências utilizadas

- **Angular Performance Optimization**
(<https://www.syncfusion.com/blogs/post/angular-performance-optimization>)
- **Melhores práticas para uma aplicação em angular limpa e de elevado desempenho**
(<https://www.freecodecamp.org/portuguese/news/melhores-praticas-para-uma-aplicacao-em-angular-limpa-e-de-elevado-desempenho/>)
- **Performance em Angular - Dicas e Truques**
(<https://medium.com/codigorefinado/performance-em-angular-dicas-e-truques-deeb6cb3860c>)

Referências utilizadas

- **Angular - Introdução aos testes**
(<https://dev.to/wesleyegberto/angular-introducao-aos-testes-2m7f>)
- **Escrevendo testes eficientes de verdade no angular**
(<https://andrewrosario.medium.com/escrevendo-testes-eficientes-de-verdade-no-angular-263a9d03cecf>)
- **Angular: testes unitários com Jasmine para componentes e serviços** (https://www.alura.com.br/artigos/angular-testes-unitarios-classes-componentes-servicos-jasmine?srsltid=AfmBOopF5DPDlPomJiDRtij3C_ZzXqtk1Xh3gUy17XFcAFs6po8_d8hk)

Muito obrigado pela atenção!

Arthur Souza e João Victor Prata
Fundação Getúlio Vargas